

git-mcp: build and verification

The third and last of the SSH-reachable MCP servers is live: typed git operations against a repo on any registered tailnet host. Full design rationale lives in `git-mcp/design-plan.pdf`, reviewed and approved before any code was written; this doc covers what actually happened building and verifying it against real infrastructure -- including two real bugs neither the design nor a local build caught.

tool: 13 typed tools, target + repo_path 2026-08-16

00 What shipped

13 typed tools, no local default target (every call names a `target` resolved against a registry), SSH-execute mechanism reusing the docker-mcp playbook exactly: dedicated keypair, reused OS accounts, ControlMaster multiplexing baked in from day one instead of discovered after a latency bug.

CATEGORY	TOOLS
Read (6)	<code>git_status</code> , <code>git_log</code> , <code>git_diff</code> , <code>git_show</code> , <code>git_branch_list</code> , <code>git_remote_list</code>
Write (7)	<code>git_add</code> , <code>git_commit</code> , <code>git_fetch</code> , <code>git_pull</code> , <code>git_checkout</code> , <code>git_branch_create</code> , <code>git_push</code>
Excluded	<code>force-push</code> , <code>reset --hard</code> , <code>clean -f</code> , <code>rebase</code> , <code>commit --amend</code> -- omitted, not gated

Live targets: `clubuntu` (self-loop) and `katmint`, both POSIX. `katlegog` (Windows) is deliberately absent from the registry -- gated pending dedicated `windows_quote()` verification, per the staged-rollout plan.

01 Implementation, in brief

Unlike docker-mcp/postgres-mcp, there was no pre-existing mechanism to discover here -- this was a from-scratch build against an approved plan. The two structural decisions that mattered most: a **target registry** instead of a free-form `ssh://` string (closes an SSH

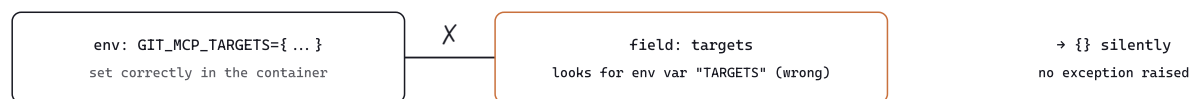
option-injection edge case, gives a fixed place to record shell dialect), and **two independent defenses** against command injection -- dialect-aware quoting (`posix_quote` / `windows_quote`) plus structural validation of ref/branch/remote shape *before* quoting ever runs. Both live in one place each (`quoting.py` , called only from `ssh_executor.py`).

Verified locally before any push: built the image standalone, ran it, confirmed all 13 tools registered with no import errors and no `force` parameter anywhere in `git_push`'s schema. Caught a Windows/Git-Bash path-mangling quirk during that local build (`/dev/null` silently became `null` , which Compose then read as a named-volume reference) -- fixed with `MSYS_NO_PATHCONV=1` , not a git-mcp bug.

02 Honest detour: the registry loaded empty

First live call after deploying `v0.1.5` : `git_status(target="clubuntu", ...)` came back `Unknown target 'clubuntu'; known targets: []` -- despite `GIT_MCP_TARGETS` being set correctly, and correctly formatted JSON, in the container's own environment.

Root cause: pydantic-settings maps a bare field name to the identically-named env var by default. The field was named `targets` , so it looked for `TARGETS` -- never `GIT_MCP_TARGETS` . The registry silently loaded as an empty dict rather than raising, because an empty dict is a perfectly valid (if useless) value for that field.



Fix: `Field(validation_alias="GIT_MCP_TARGETS")` -- explicit, not inferred

Deployed as `v0.1.6`. Confirmed via `docker exec git-mcp python -c "... settings.targets ..."` before re-testing the MCP call.

03 Proof: real repos, three angles

This session's Claude Code tool cache didn't know about the newly-registered `git` MCP server mid-session -- the same stale-cache situation documented in §04 of the postgres-mcp doc. Verification continued via direct MCP protocol calls (manual initialize handshake, session ID, `tools/call`) instead of waiting for a fresh session.

```
GIT_STATUS(TARGET="CLUBUNTU", REPO_PATH="/OPT/DOCKER/MANPAGE")

On branch main
Changes not staged for commit:
    modified:   ../homepage/config/services.yaml
Untracked files:
    ../caddy/  ../core/  ../dbs/  ...
```

Real output, real repo -- and it revealed something about clubuntu's own layout worth knowing: `/opt/docker/manpage`'s `.git` root actually covers the whole `/opt/docker/` tree (every docker project on that host shares one parent repo), not something git-mcp caused, just something git-mcp's honesty surfaced.

```
GIT_LOG(TARGET="KATMINT", REPO_PATH=".../WORK") -- SCRATCH REPO

4f76830 (HEAD → master, origin/main) init
```

04 Proof: injection defenses, live

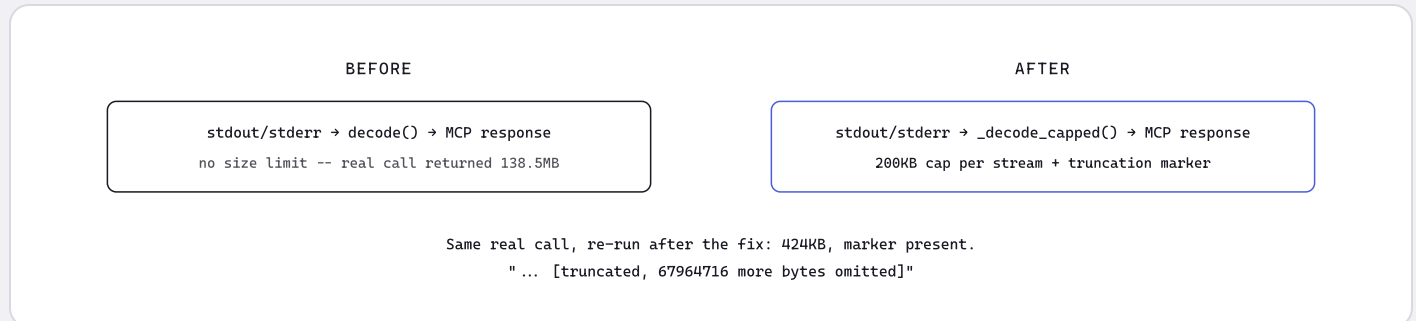
Not unit tests -- real calls against a disposable scratch repo on katmint, through the full gateway → git-mcp → ssh → git path, with a server-side check afterward for exactly what an attacker would be hoping for.

ATTEMPT	RESULT
Commit message: <code>test `id` and \$(whoami) and ; rm -rf /tmp/pwned ; echo done</code>	✓ stored as inert literal text in the commit -- confirmed via <code>git log --format=%s ; /tmp/pwned</code> never created
Branch name: <code>--upload-pack=touch /tmp/pwned2</code>	✓ rejected by <code>validate_ref_name</code> before reaching the shell: "must not start with '-"; <code>/tmp/pwned2</code> never created

Both defenses -- dialect-aware quoting for the first, structural pre-validation for the second -- did exactly what §03 of the design plan said they would, against a live attempt, not a hypothetical one.

05 Honest detour: 138.5MB from one diff

Rounding out read-tool coverage, `git_diff(target="clubuntu", repo_path="/opt/docker/manpage")` -- no arguments beyond the required two -- came back **138.5MB**. That parent repo (see §03) tracks a log file that had grown to 1.6M lines. Nothing in `ssh_executor.py` bounded output size; a caller-supplied `repo_path` / `ref` can hit arbitrarily large real content, and this wasn't a hypothetical, it happened on the first real-repo diff call.



Deployed as v0.1.7. The cap bounds the response returned to the MCP client; it does not bound subprocess memory during `communicate()` itself, which still buffers the full output before this function sees it -- an explicitly flagged, not silently ignored, limitation.

06 Deploy history

TAG	CHANGE
v0.1.5	Initial git-mcp: 13 tools, SSH-execute, target registry, two-layer injection defense
v0.1.6	Fix: target registry silently loaded empty (§02)
v0.1.7	Fix: unbounded output size, found via a real 138.5MB response (§05)

Both fixes were found by testing against real, live infrastructure after deploy -- neither surfaced in the local build-and-smoke-test pass, and neither was anticipated in the design plan. That gap between "builds and imports cleanly" and "behaves correctly against real data and a real registry" is exactly why the verification plan called for live testing rather than stopping at a clean build.

07 Where this leaves things

- ▶ All three SSH-reachable MCP servers (docker-mcp, postgres-mcp's connection_uri path, git-mcp) are now deployed, proven against real infrastructure, and documented.
- ▶ `katlegog` stays out of `GIT_MCP_TARGETS` until `windows_quote()` gets its own dedicated verification pass against real cmd.exe metacharacters -- the one piece of the original design plan deliberately not exercised yet.
- ▶ A fourth target host (or a fourth MCP server) now has two proven templates to follow: docker-mcp's for SSH-executed tools in general, and this doc's two detours as a reminder that "builds clean" and "correct under real registry/data conditions" are different bars.

final-run - 6 · git-mcp: 13 tools, 2 targets live, 2 real bugs found and fixed against production, 0 force operations anywhere